



昇腾950特性全景剖析



演讲人 李东锋

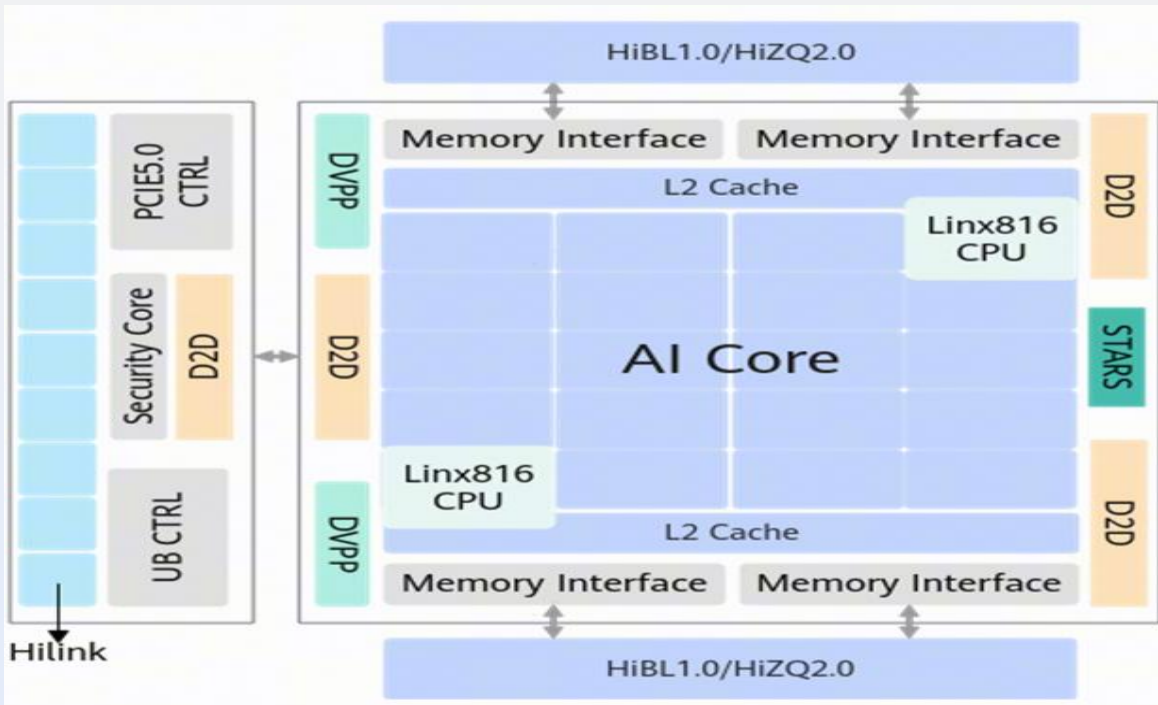


< 鲲鹏昇腾开发者大会2026 >

Ascend 950 芯片架构

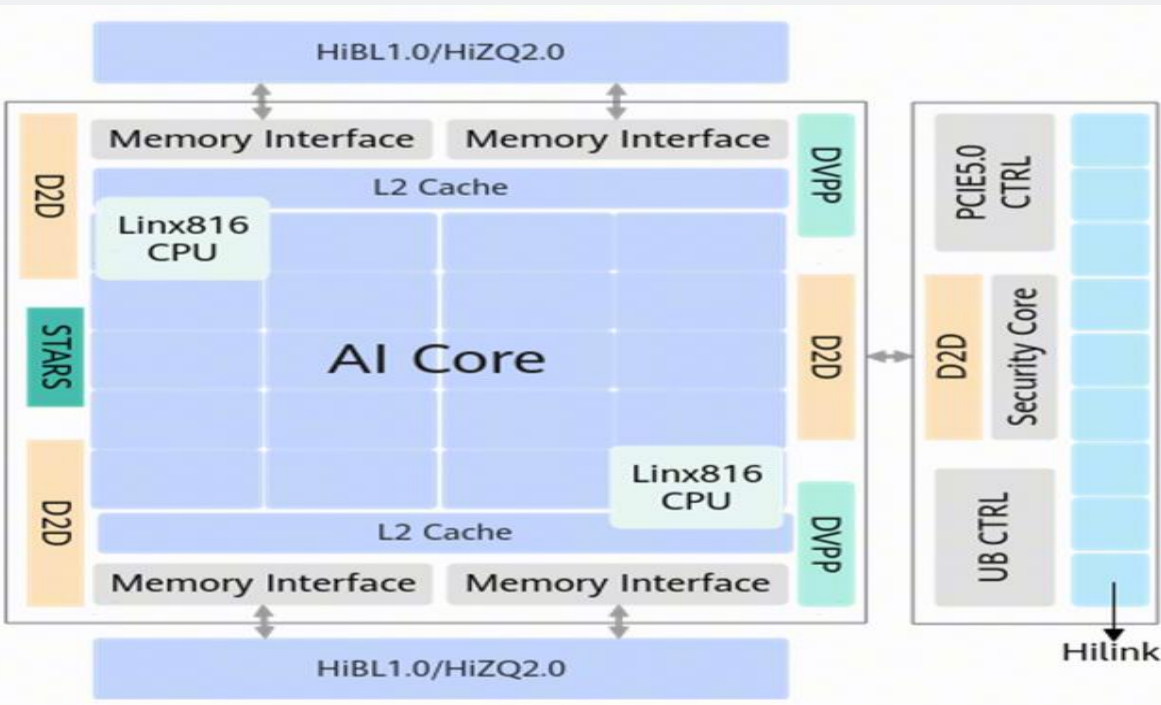
Ascend 950PR: 中等算力/带宽, 低成本

适用场景: LLM-Prefill、推荐、多模态



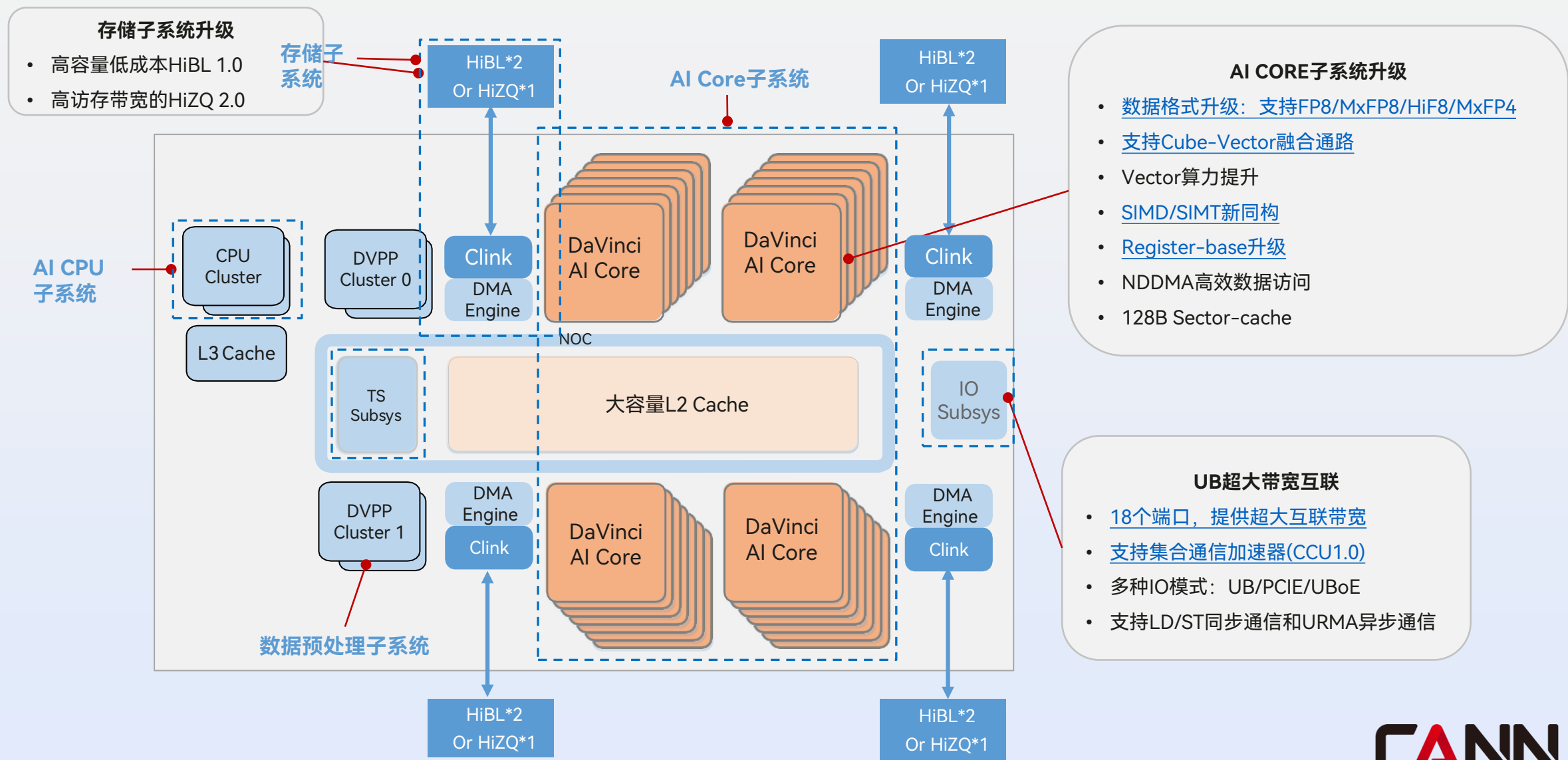
Ascend 950DT: 高算力/带宽, 中等成本

适用场景: LLM-Decode、训练

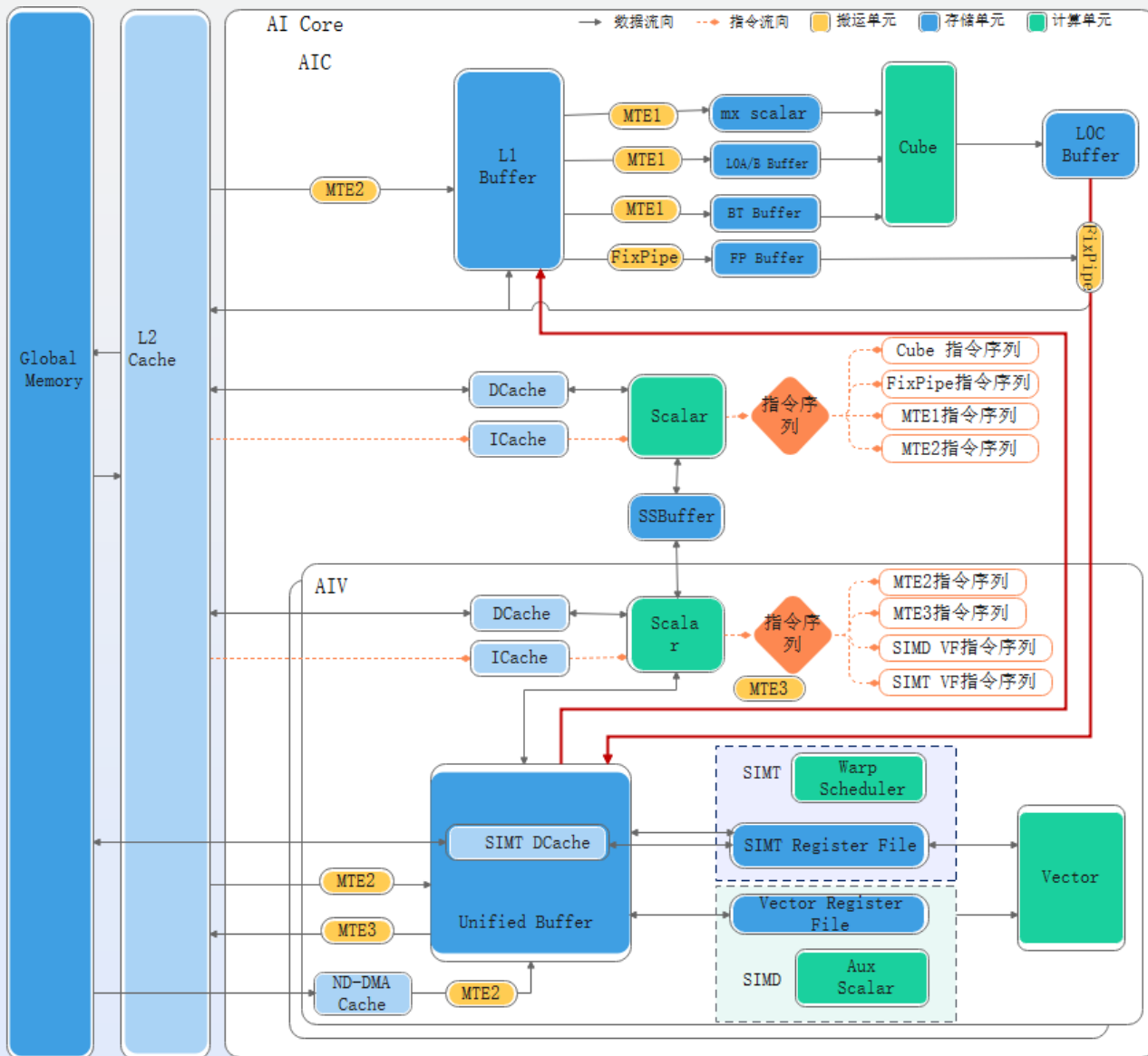


- 多DIE合封(计算DIE、2 * IO DIE) , 统一接口, 合封不同Memory
- D2D大带宽, 双DIE UMA, 易用性提升

Ascend 950 芯片架构



Ascend 950 微架构框图



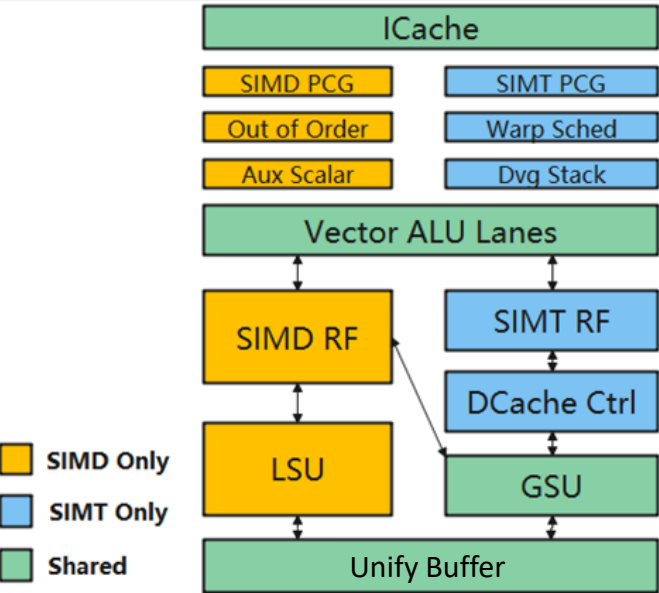
Cube MMAD计算支持的数据类型:

1. FP16/BF16/HF32/FP32/S8
2. FP8(E5M2/E4M3)
3. MXFP8(E5M2/E4M3)
4. MXFP4(E2M1/E1M2)
5. HiF8

950相比910B/910C数据通道主要改动点:

1. 增加LOC ->UBuffer数据通路
2. 增加UBuffer->L1数据通路
3. 增加SSBuf支持CV之间的消息通路

Ascend 950 Vector单元SIMT/SIMD



SIMT Only 组件:

- 1. Warp Schedule -多线程调度
- 2. Divergence Stack -支持分支调度
- 3. SIMT RF – SIMT专用Register File
- 4. DCache – SIMT数据缓存

SIMD 和 SIMT 共享组件:

- 1. ICache – 指令Cache
- 2. ALU – 计算单元
- 3. Unify Buffer – SIMT Share memory和 SIMD复用
- 4. GSU – Gather Scatter Unit

规格名称	SIMT	SIMD
编程范式	标量编程	矢量编程(需要感知硬件并行度)
控制逻辑	硬件支持非结构化控制逻辑下的并行	控制逻辑与并行之间的关系需要软件处理
GM离散访问	支持直接访问GM; 有核内DCache, 对离散访问较友好	不支持直接访问GM
指令发射效率	每个Cycle发射单指令	每个Cycle支持同时发射数据加载到寄存器、计算、写回到UB操作
UB访存效率	UBuffer:R/W x B/s	UBuffer: 4*x B/s

Ascend 950 SIMT实现的GatherV2样例 - 标量计算

```
61 template <typename X_T, typename INDICES_T, typename INDEX_SIZE_T>
62 template <const bool NIS>
63 __simt_vf__ __aicore__ LAUNCH_BOUND(THREAD_NUM_LAUNCH_BOUND) inline void GatherV2<X_T, INDICES_T, INDEX_SIZE_T>::GatherSimt(const INDEX_SIZE_T yIndexBase,
64 INDEX_SIZE_T currentCoreElements, INDEX_SIZE_T m0, INDEX_SIZE_T shift0, INDEX_SIZE_T m1, INDEX_SIZE_T shift1, INDEX_SIZE_T m2,
65 INDEX_SIZE_T shift2, INDEX_SIZE_T gatherSize, INDEX_SIZE_T outerSize, INDEX_SIZE_T innerSize,
66 INDEX_SIZE_T gatherDimSize, __gm__ X_T* x, __gm__ INDICES_T* indices, __gm__ volatile X_T* y) {
67     for (INDEX_SIZE_T index = static_cast<INDEX_SIZE_T>(Simt::GetThreadId()); index < currentCoreElements;
68         index += static_cast<INDEX_SIZE_T>(Simt::GetThreadNum())) {
69         INDEX_SIZE_T yIndex = yIndexBase + index;
70
71         INDEX_SIZE_T tmp = Simt::UIntDiv(yIndex, m0, shift0);
72         INDEX_SIZE_T tmp1 = Simt::UIntDiv(tmp, m1, shift1);
73         INDEX_SIZE_T gatherI = tmp - tmp1 * gatherSize;
74         INDEX_SIZE_T batchI = Simt::UIntDiv(tmp1, m2, shift2);
75
76         INDEX_SIZE_T outerI = tmp1 - batchI * outerSize;
77         INDEX_SIZE_T innerI = yIndex - tmp * innerSize;
78         INDEX_SIZE_T indicesIndex = batchI * gatherSize + gatherI;
79
80         INDICES_T indicesValue = indices[indicesIndex];
81         if constexpr(NIS) {
82             if (unlikely(indicesValue < 0)) {
83                 indicesValue += gatherDimSize;
84             }
85         }
86         INDEX_SIZE_T indicesValueI = static_cast<INDEX_SIZE_T>(indicesValue);
87         INDEX_SIZE_T xIndex = (tmp1 * gatherDimSize + indicesValueI) * innerSize + innerI;
88         bool idxOutOfBounds = indicesValue < 0 || indicesValueI >= gatherDimSize;
89         y[yIndex] = idxOutOfBounds ? 0 : x[xIndex];
90     }
91 }
```

计算各输出轴的Indices

计算Gather轴上Indices的位置

获取到Gather轴上Indices的数值

Indices为负值处理

根据输入Indices执行Gather操作

Ascend 950 SIMT实现的InsertHash样例 - 多条件分支

```
145 while (false == insertSuccess) {
146     if (counter++ >= tableSize) {
147         break;
148     }
149     const int32_t originalFlag = Simt::AtomicCas(
150         reinterpret_cast<__gm__ int32_t*>(table + tableOffset + TABLE_FLAG_OFFSET), static_cast<int32_t>(0),
151         BIG_ENDIAN_ONE);
152
153     if (0 == originalFlag) {
154         *reinterpret_cast<__gm__ int64_t*>(table + tableOffset) = insertKey;
155         Simt::ThreadFence();
156         *reinterpret_cast<__gm__ int32_t*>(table + tableOffset + TABLE_STATE_OFFSET) = 1;
157         insertSuccess = true;
158         Simt::AtomicAdd(tableHandleAddr + HANDLE_SIZE_ALL_OFFSET, INT64_ONE);
159         Simt::AtomicAdd(tableHandleAddr + HANDLE_SIZE_ALL_NOEXPORT_OFFSET, INT64_ONE);
160         break;
161     } else {
162         while (*reinterpret_cast<__gm__ volatile int32_t*>(table + tableOffset + TABLE_STATE_OFFSET) != 1) {
163         }
164         int64_t currentKey = *reinterpret_cast<__gm__ volatile int64_t*>(table + tableOffset);
165         if (currentKey == insertKey) {
166             auto flag = *reinterpret_cast<__gm__ volatile int32_t*>(table + tableOffset + TABLE_FLAG_OFFSET);
167
168             if ((flag & EVICTED_FLAG_MASK) != 0) {
169                 auto newFlag = flag ^ EVICTED_FLAG_MASK;
170                 auto oldFlag = Simt::AtomicCas(
171                     reinterpret_cast<__gm__ int32_t*>(table + tableOffset + TABLE_FLAG_OFFSET),
172                     static_cast<int32_t>(flag), newFlag);
173
174                 if ((oldFlag & EVICTED_FLAG_MASK) != 0) {
175                     Simt::AtomicAdd(tableHandleAddr + HANDLE_SIZE_ALL_OFFSET, INT64_ONE);
176                     Simt::AtomicAdd(tableHandleAddr + HANDLE_SIZE_ALL_NOEXPORT_OFFSET, INT64_ONE);
177                 }
178             }
179         }
180     }
181 }
```

遍历Table表

Key与计划插入位置数值执行原子粒度的Compare And Swap操作

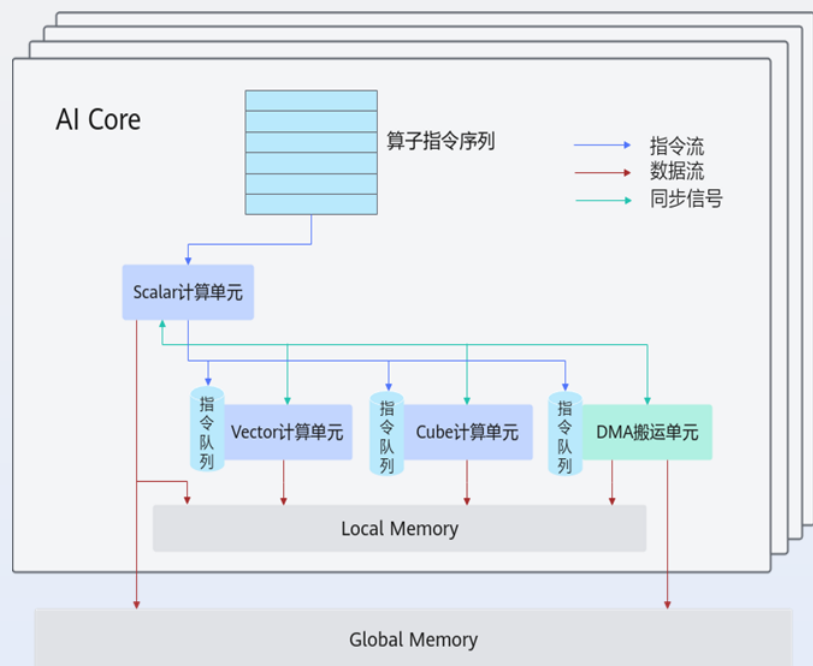
本线程插入Key成功

更新Table Handle

Key已经被成功插入过

Ascend 950 Vector单元SIMD-Regbase架构

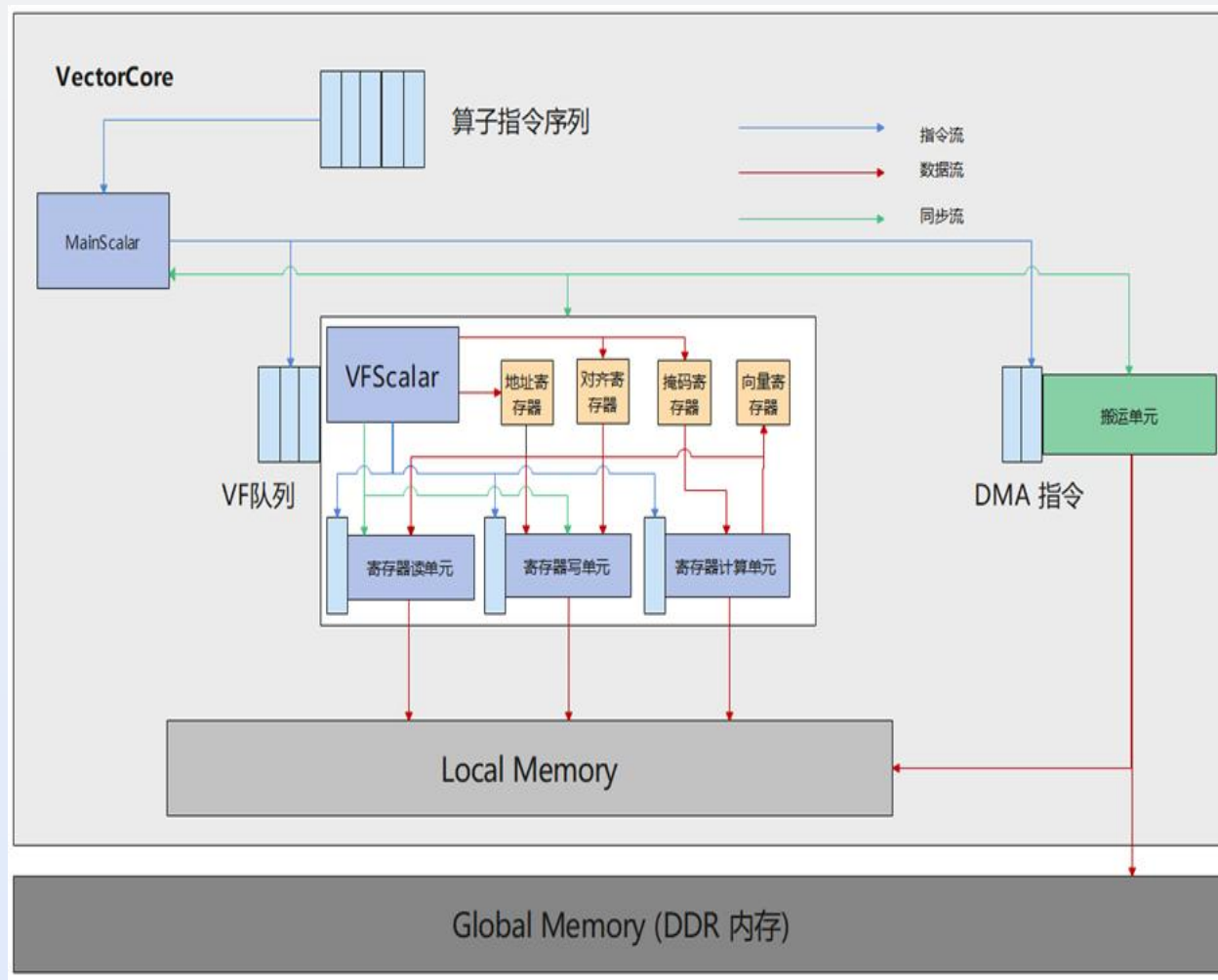
Membase



Regbase 架构优点:

1. 寄存器内计算，减少对Local buffer访问带宽；
2. 通过支持OOO实现指令双发，提升Vector性能；
3. 便于支持Local buffer非32B对齐数据处理；

Regbase



Ascend 950 Regbase 编程样例

```
__aicore__ inline RsqrtCustom(LocalTensor<T> &dst, LocalTensor<T> &src, uint32_t count) {
    uint32_t dtypeSize = sizeof(T);
    uint32_t VL = AscendC::VECTOR_REG_WIDTH / dtypeSize;
    uint32_t loopNum = (count + VL - 1) / VL;
    uint32_t vlSize = VL;

    __VEC_SCOPE__ {
        __ubuf__ T* srcAddr = (__ubuf__ T*)src.GetPhyAddr();
        __ubuf__ T* dstAddr = (__ubuf__ T*)dst.GetPhyAddr();

        static constexpr AscendC::MicroAPI::DivSpecificMode mode = {AscendC::MicroAPI::MaskMergeMode::ZEROING, true};

        AscendC::MicroAPI::RegTensor<T, AscendC::MicroAPI::RegTraitNumOne> vregInput;
        AscendC::MicroAPI::RegTensor<T, AscendC::MicroAPI::RegTraitNumOne> ones;
        AscendC::MicroAPI::RegTensor<T, AscendC::MicroAPI::RegTraitNumOne> vregSqrt;
        AscendC::MicroAPI::RegTensor<T, AscendC::MicroAPI::RegTraitNumOne> vregOutput;
        AscendC::MicroAPI::MaskReg mask;

        for (uint16_t loopIdx = 0; loopIdx < loopNum; loopIdx++) {
            mask = AscendC::MicroAPI::UpdateMask<T, AscendC::MicroAPI::RegTraitNumOne>(count);
            // OpCopyIn0
            AscendC::MicroAPI::DataCopy(vregInput, (__ubuf__ T*)(srcAddr + loopIdx * vlSize));
            // compute rsqrt
            AscendC::MicroAPI::Duplicate(ones, (T)1.0, mask);
            AscendC::MicroAPI::Sqrt(vregSqrt, vregInput, mask);
            // low precision mode(default)
            //AscendC::MicroAPI::Div(vregOutput, one, vregSqrt, mask);
            // high precision mode div
            AscendC::MicroAPI::Div<T, &mode>(vregOutput, ones, vregSqrt, mask);

            // OpCopyOut
            AscendC::MicroAPI::DataCopy((__ubuf__ T*)(dstAddr + loopIdx * vlSize), vregOutput, mask);
        }
    }
}
```

Regbase相比Membase编程方式区别:

1. 显式的Register数据读写;
2. 处理的数据由Membase的block, repeat等参数换成的for-loop的形式, 代码更加容易编写和理解

Ascend 950 Chiplet和IO接口

Ascend 950相比上一代芯片从2Die to 4Die, 把Compute与IO进行了切割

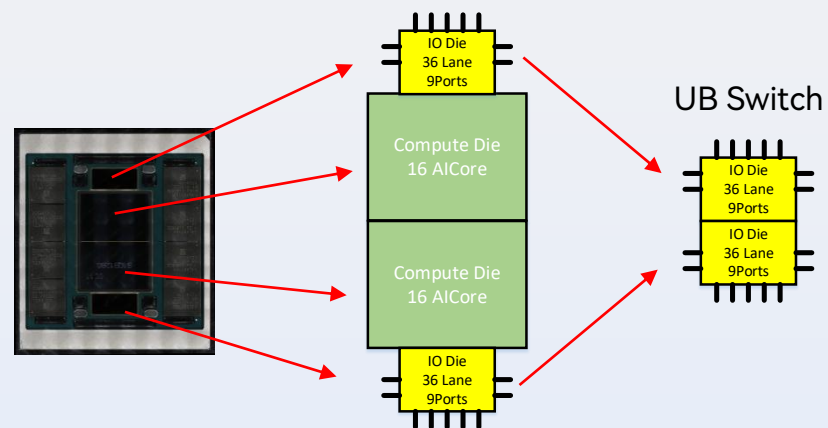
- 1、切割后可控制Die Size及良率
- 2、IO-Die与工艺相关度弱, 可独立演进, 也可自行拼装为Switch

IO-Die结构:

- 1、36 Lane 按1个port x4 Lane分成9个Port
- 2、支持9 x4 UB端口(两个IO-Die支持最大18 x4)
- 3、支持 x4 UBoE, 即UB over Ethernet 技术;
- 4、支持 x16 PCIe, Ascend 950可以对接PCIe Device

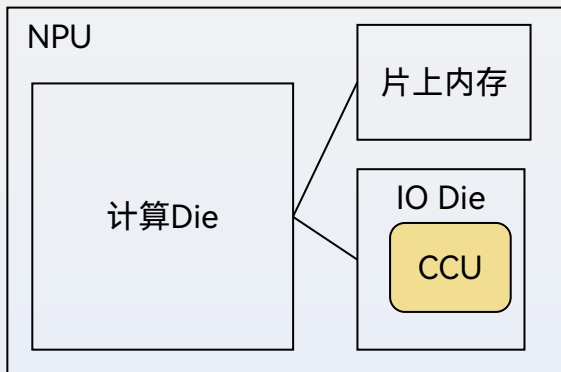
集合通信单元CCU:

- 1、CCU位于IO-Die内
- 2、通过软件预置的集合通信指令, 由CCU硬件完成通信节点间的同步、地址交换、数据传输以及归约操作, 加速集合通信

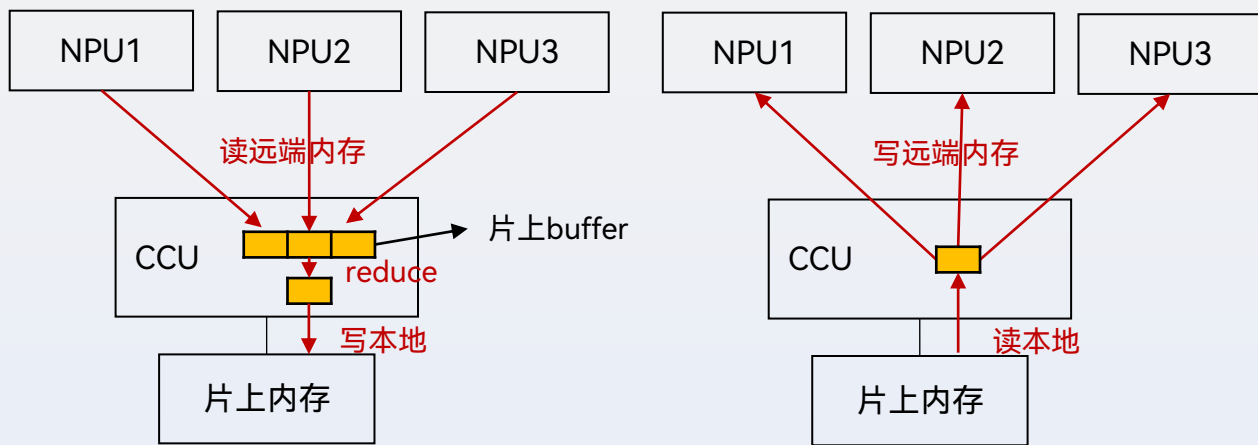


专用通信引擎CCU - 降低内存带宽占用、降低通信时延、天然零拷贝及确定性计算

Collective Communication Unit (CCU) 是Ascend 950上新增的用于加速集合通信的专用引擎，CCU根据软件预置的集合通信算法指令，完成通信节点间的同步、地址交换、数据传输以及规约计算，实现集合通信操作



CCU位置示意图

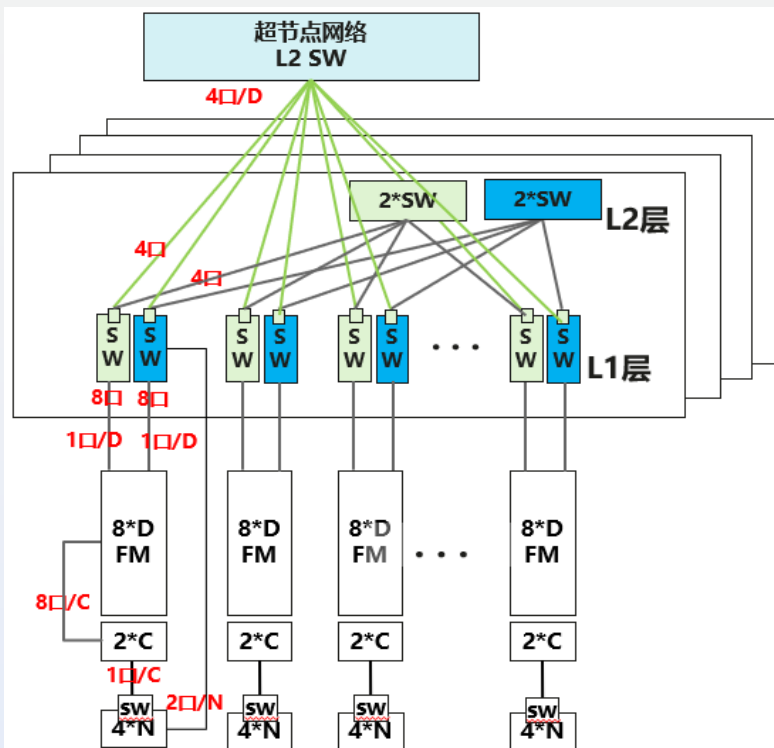


CCU 核心能力

解决的问题及优势:

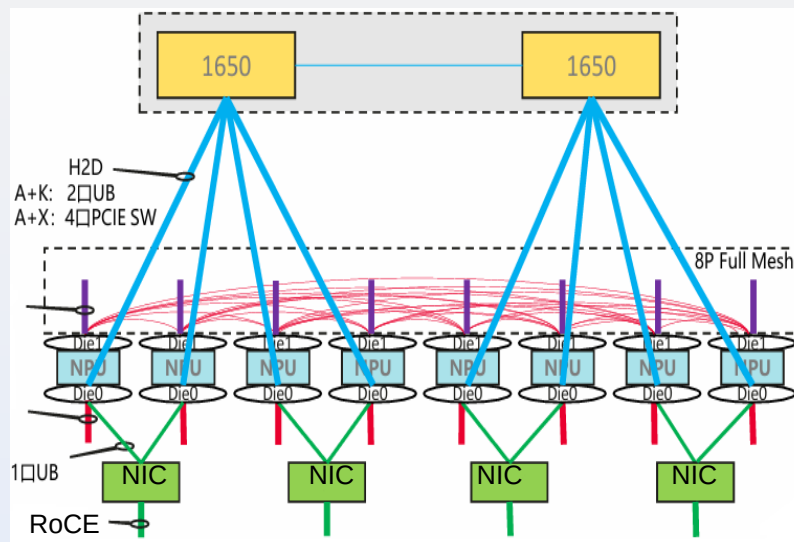
- 1、利用CCU片上buffer缓存通信数据，降低访存带宽需求
- 2、利用片上buffer+计算单元顺序执行Reduce操作，确保计算保序
- 3、专用的通信调度与同步机制，降低时延，不占用计算算力
- 4、消减用户buffer到CCL Buffer的拷贝

Ascend 950 硬件形态: POD & Server & 标卡



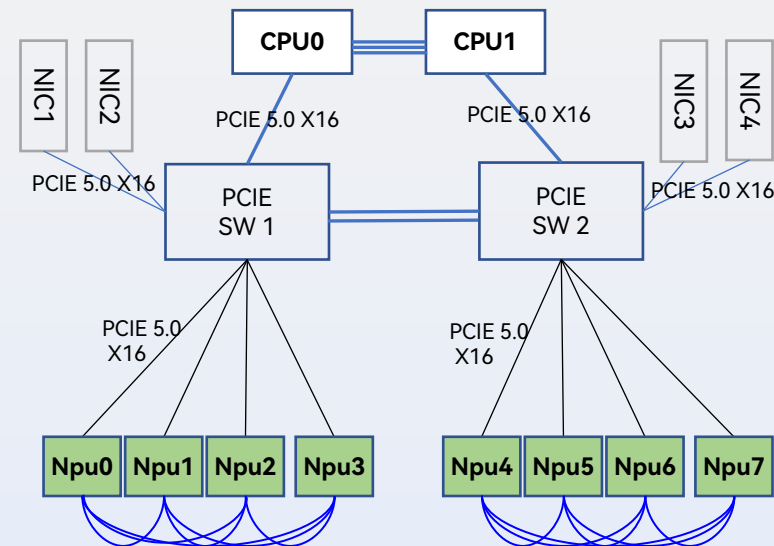
POD形态

- ① 1D fullmesh+clos拓扑, 64 NPU + 16 CPU
- ② 计算板: 每D出7*X4 UB 端口组成Fullmesh连接, 每D出8口对接L1 SW
- ③ 交换板: 通过不同的L2 SW配置, 可以等效支持每D出4个端口或者8端口, 作为框内板间互联或者框间互联



Server形态

- ① 8 NPU + 2 CPU, CPU节点支持鲲鹏及X86
- ② server内: Fullmesh拓扑, 7*X4 UB口
- ③ 出server: Clos拓扑, 8*X4 UB口



标卡形态

- ① 基于X86底座, 支持四卡UB互联
- ② 每卡3个X4 UB端口组成Fullmesh连接
- ③ 可接第三方网卡, 做参数面互联方案

Ascend 950超节点采用POD& Server作为基本互联单位，通过灵衢互联构建高性能集群

- 1、超节点内CPU、NPU、存储等异构组件采用无主从直接通信，实现组件对等协同；
- 2、超节点内协议归一，所有节点都使用一套协议互连互通；
- 3、超节点内原生支持内存语义，兼顾异步访问的消息语义，既保证了极致性能，又兼顾兼容性；

灵衢互联特征(UB)

对等架构

打破以CPU为中心的互联模式

统一互联协议

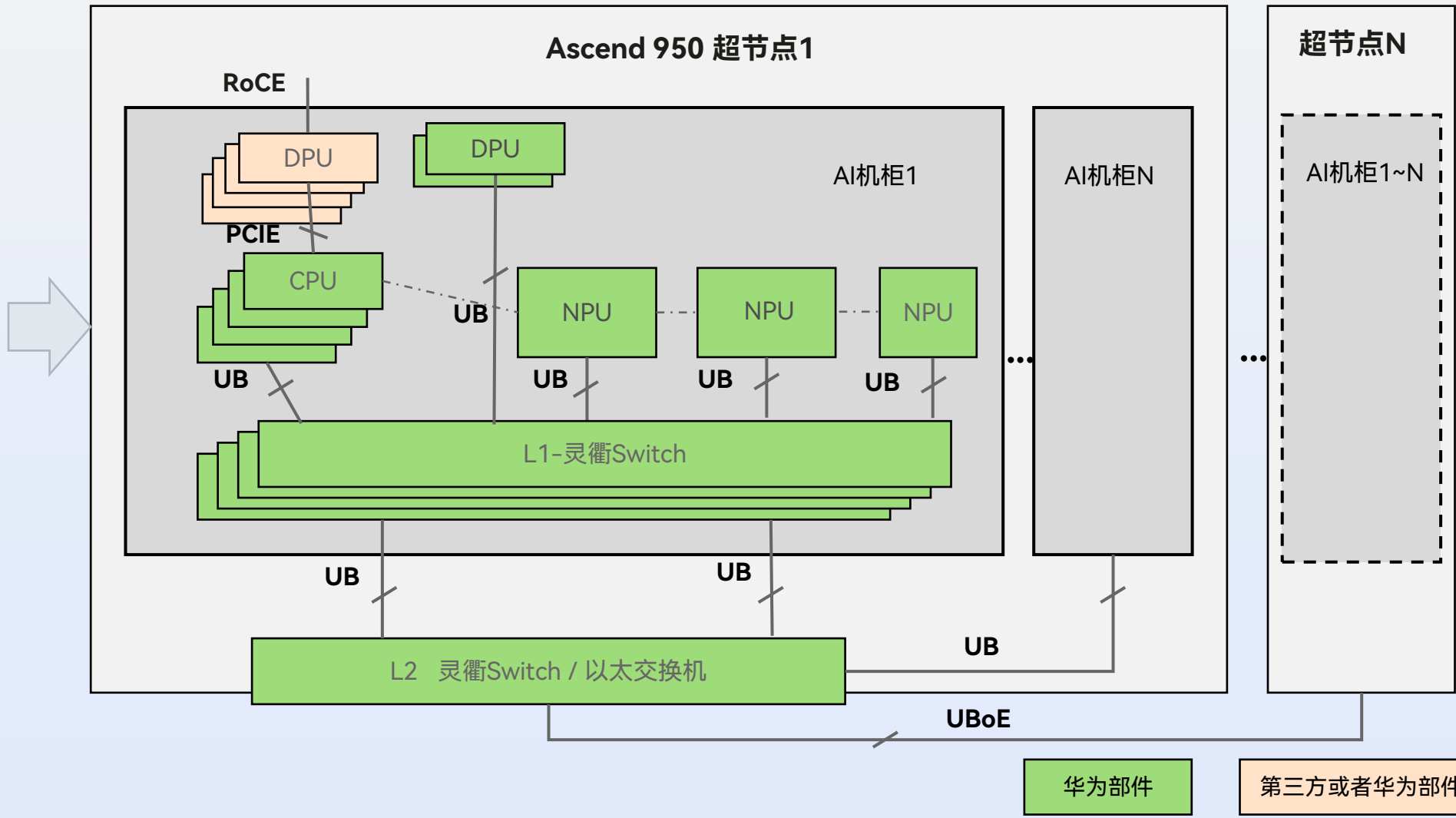
UB 统一互联协议，避免协议转换

高性能，大规模

低时延，大带宽

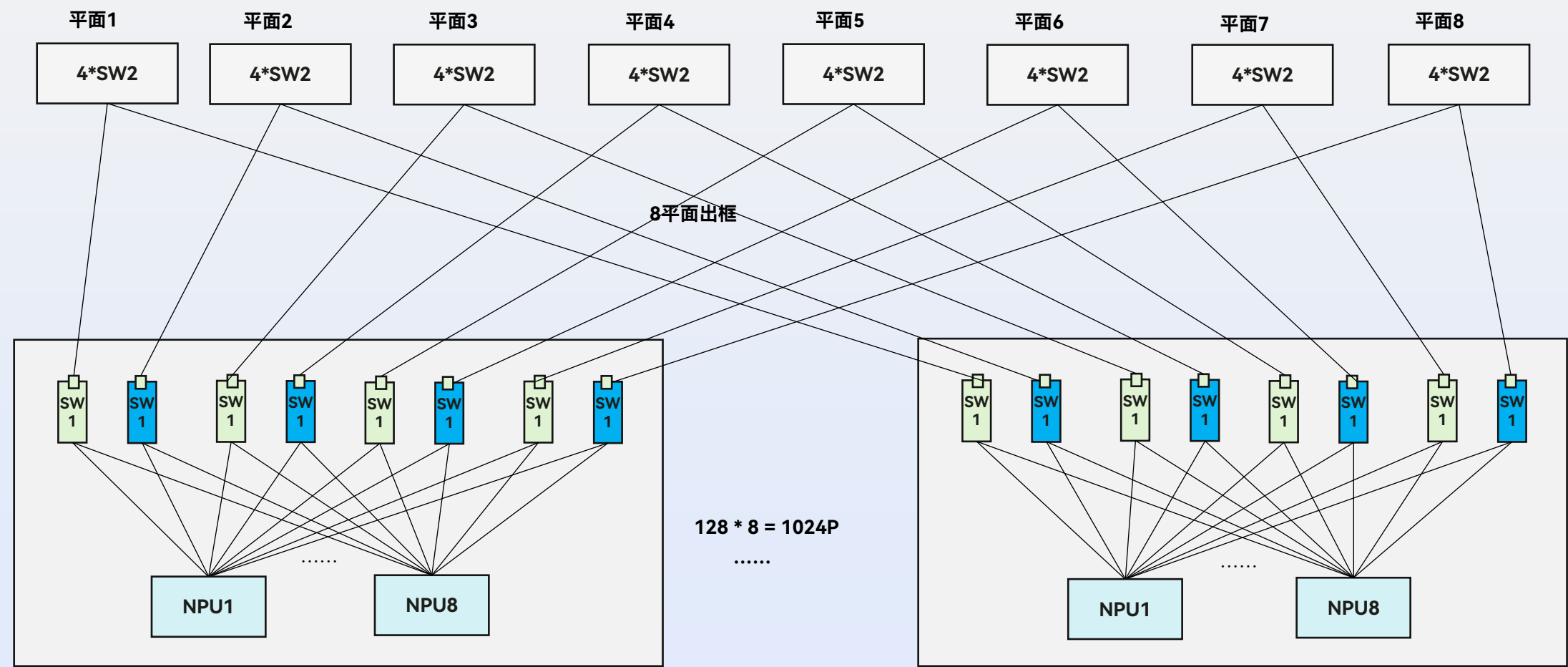
全域内存语义

采用内存地址访问，降低编程复杂度



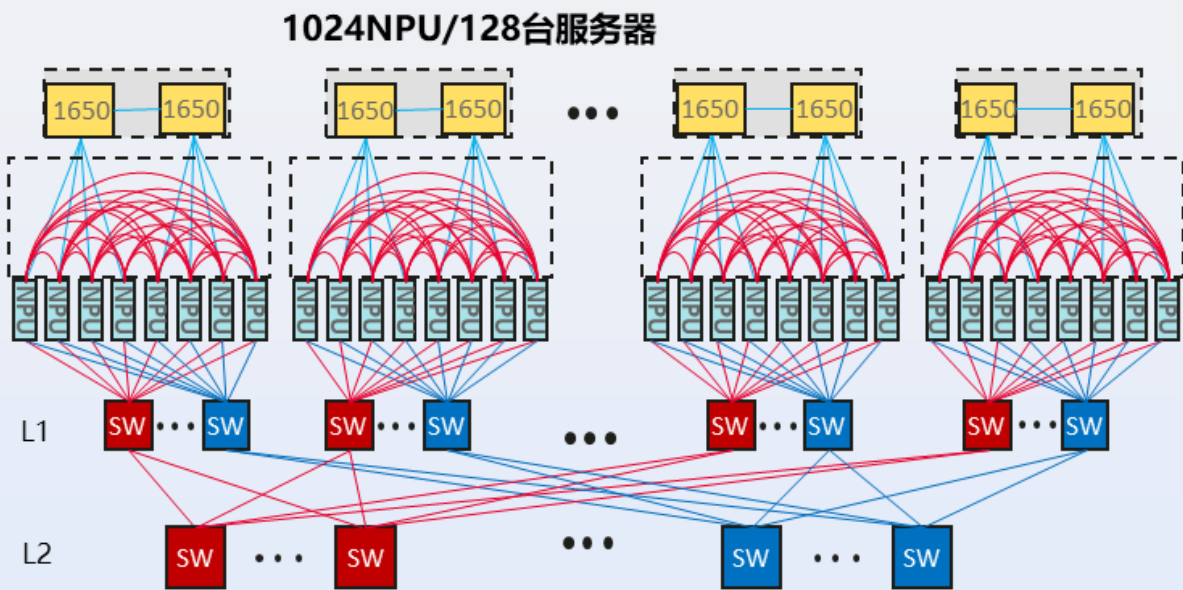
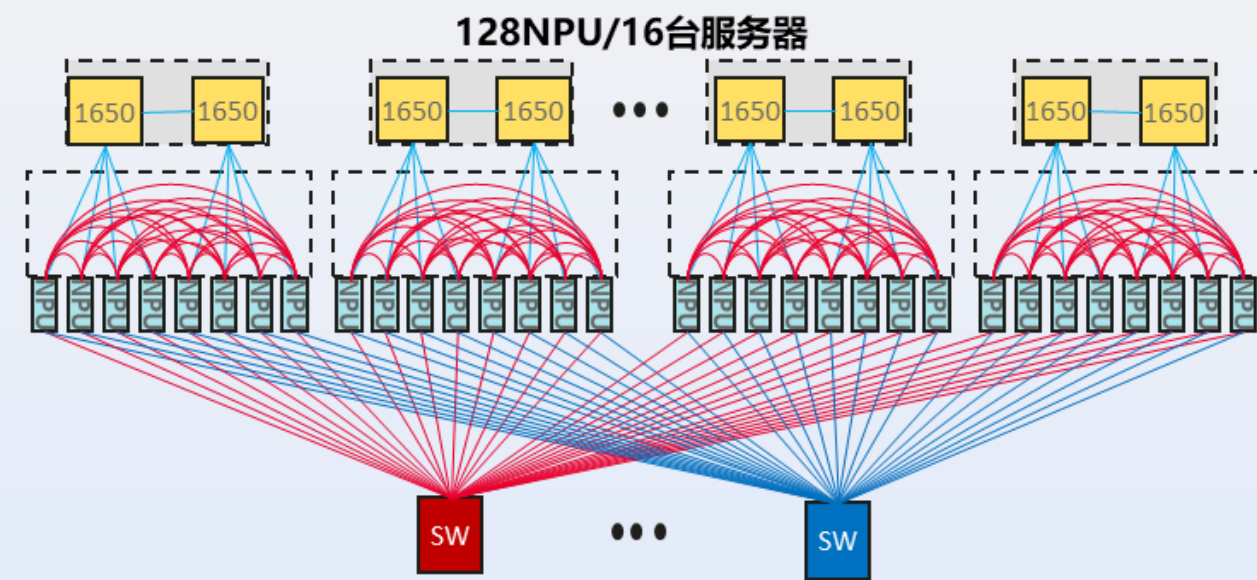
Ascend 950 POD间使用单层SW组网构建1024P超节点集群

- 1、Ascend 950 POD可以采用出框单层SW构建SuperPOD超节点;
- 2、L1 SW和L2 SW构成Clos互联;
- 3、多个独立平面，平面间无互联;



Ascend 950 server使用单层或者两层SW构建多种规格超节点

- 1、128卡超节点：通过一层SW设备实现16台服务器，128张卡的对等全互联；
- 2、1K超节点：通过L1 SW先构建服务器内8张卡接入、L2 SW实现TOR的互联汇聚；
- 3、多个独立平面，平面间无互联；



Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯

<https://gitcode.com/cann>

CANN